

Sistema de Leitura RFID via MQTT

Projeto de Automação II

Tomás Espincho 110486

Universidade de Aveiro

17 de junho de 2026

- 1 Introdução e Objetivos
- 2 Arquitetura do Sistema
- 3 Hardware
- 4 Firmware ESP32
- 5 Protocolo MQTT e Broker
- 6 Scripts Python
- 7 Formato de Dados

Objetivo Principal

Desenvolver um projeto capaz de **ler tags RFID** e **transmitir dados em tempo real** via MQTT.

Objetivos Específicos

- Capturar o UID de cartões RFID com ESP32
- Sincronizar tempo via NTP
- Publicar dados JSON no broker MQTT
- Receber e processar os dados em Python
- Simular leituras sem hardware físico

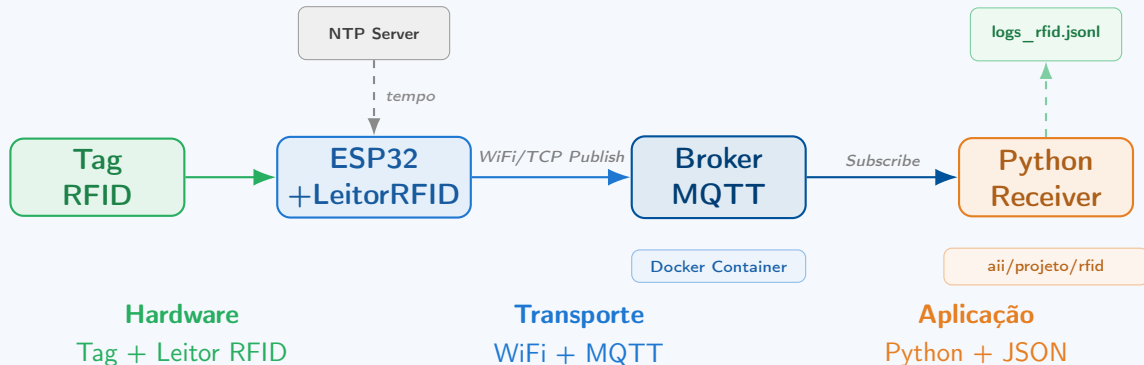
Casos de Uso

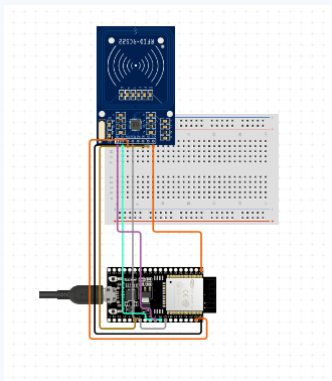
Controlo de Acessos

Registo de Presenças

Inventário Industrial

Arquitetura do Sistema





Função das Entradas RFID

- **3.3V / GND:** Alimentação.
- **RST (Pino 0):** Reset físico e *power-down*.
- **SDA/SS (Pino 5):** Seleção do Slave SPI.
- **SCK (Pino 18):** Sinal de Clock do SPI.
- **MISO (Pino 19):** Saída de dados do leitor para o ESP32.
- **MOSI (Pino 23):** Entrada de dados do ESP32 para o leitor.

Firmware ESP32 – Leitura e Publicação

Excerto do Loop Principal

```
1 // 1. Ler e formatar UID (ex: "
    A1B2C3D4")
2 String tagUID = readAndFormatUID()
    ;
3
4 // 2. Obter Timestamp (NTP)
5 unsigned long ts = timeClient.
    getEpochTime();
6
7 // 3. Construir Payload JSON
8 String payload = "{\"uid\": \"" +
    tagUID +
9 "\", \"timestamp\": " + String(ts)
    + "\"}";
10
11 // 4. Publicar no Broker
12 client.publish("aii/projeto/rfid",
    payload.c_str());
```

Fluxo do programa



Protocolo MQTT e Infraestrutura

O que é o MQTT?

Message Queuing Telemetry Transport

- Protocolo **pub/sub** leve para IoT
- Baixo consumo de banda e energia
- Baseado em **tópicos** hierárquicos

Tópico Utilizado

`aii/projeto/rfid`

Publisher: ESP32 | Subscriber: Python

Broker – Eclipse Mosquitto

- Utilizado para testes locais
- Executado localmente via **Docker**

`docker-compose.yml`

```
services:
  mosquitto:
    image: eclipse-mosquitto:
      latest
    ports:
      - "1883:1883"
    volumes:
      - ./conf/mosquitto.conf:
        /mosquitto/config/
        mosquitto.conf
```

Scripts Python – Recetor e Simulador

reciver.py – Recetor MQTT

```
1 def on_message(client, userdata, msg):
2     dados = json.loads(msg.payload.decode())
3     print(f"TAG LIDA: {dados.get('uid')}")
4
5     # Setup e Subscricao
6     client = mqtt.Client()
7     client.on_message = on_message
8     client.connect(BROKER, PORT)
9     client.subscribe("aai/projeto/rfid")
10    client.loop_forever()
```

sender.py – Simulador

```
1 while True:
2     payload = json.dumps({
3         "uid": "A1B2C3D4",
4         "timestamp": int(time.time())
5     })
6
7     client.publish("aai/projeto/rfid", payload)
8     time.sleep(random.randint(3,8))
```

Configuração .env

```
MQTT_BROKER=192.168.1.100
MQTT_PORT=1883
```


Formato de Dados – JSON

Payload do ESP32 (hardware)

```
{  
  "uid": "A1 B2 C3 D4",  
  "timestamp": "1748900000"  
}
```

Persistência

O ficheiro logs_rfid.jsonl pode ser ativado no recetor para guardar todas as leituras em disco.

Payload do Simulador Python

```
{  
  "dispositivo": "Esp32",  
  "uid": "A1B2C3D4",  
  "status": "lido",  
  "timestamp": 1748900000  
}
```

Conclusão e Próximos Passos

O que foi implementado

- ✓ Leitura de tags RFID (ESP32 + MFRC522)
- ✓ Sincronização de tempo via NTP
- ✓ Publicação de JSON via MQTT
- ✓ Broker local com Docker (Mosquitto)
- ✓ Recetor Python
- ✓ Simulador para testes sem hardware

Possíveis Extensões

- Base de dados (InfluxDB / SQLite)
- Dashboard (Grafana / Node-RED)
- Controlo de acesso com lista de UIDs
- Autenticação MQTT com TLS
- Múltiplos leitores em paralelo

Tag RFID → ESP32 → MQTT → Python

O que vamos mostrar

- 1 Recetor Python — Escuta de mensagens no tópico `aii/projeto/rfid`
- 2 ESP32 + Leitor RFID — Leitura de tags físicas e publicação de dados
- 3 Simulador Python — Injeção de leituras virtuais

Obrigado!

Projeto de Automação II

Questões?